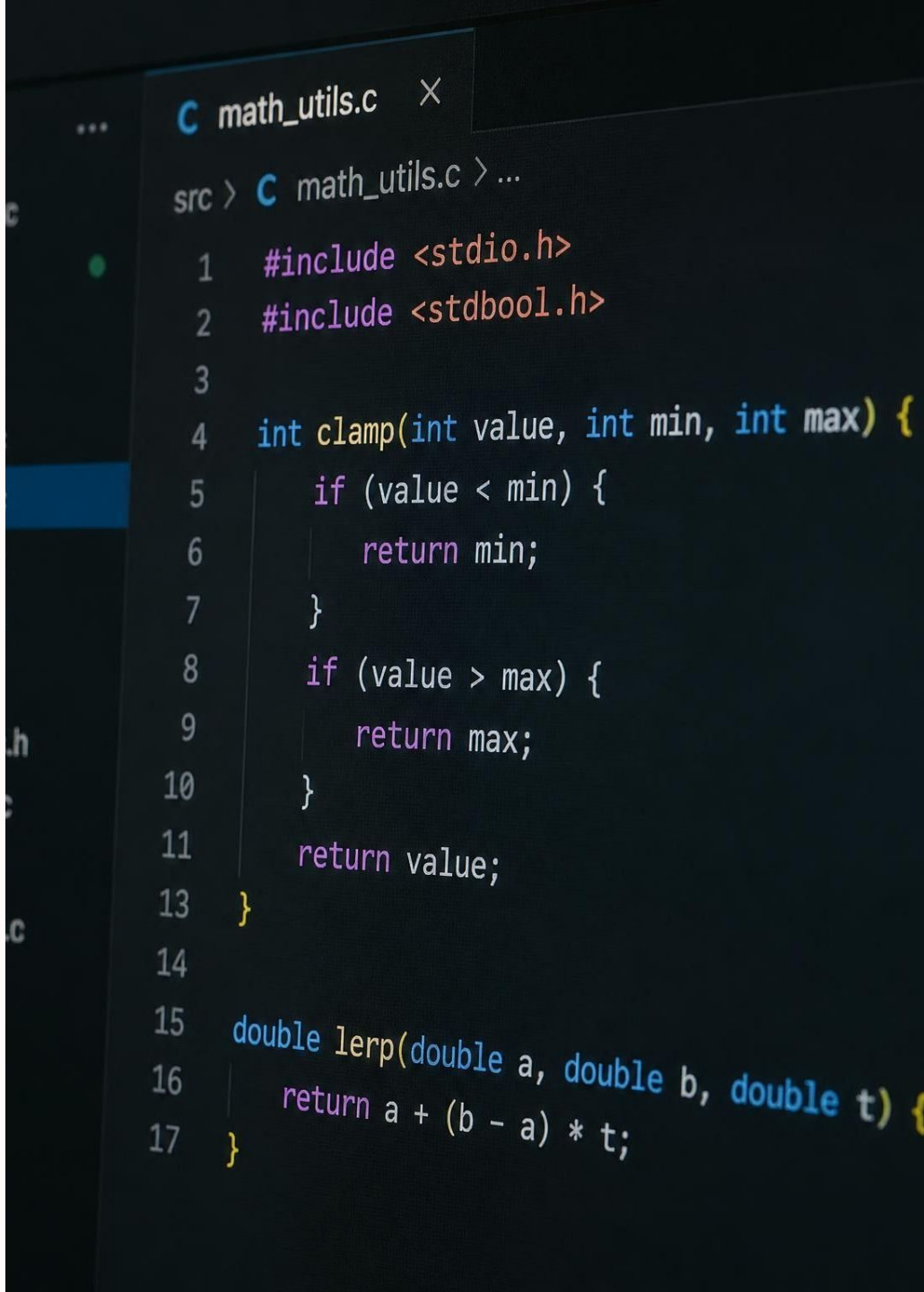
Le Funzioni in C

Definizione e chiamata, parametri e valore di ritorno, passaggio per riferimento, scope e visibilità

LINGUAGGIO C | FUNZIONI | PROGRAMMAZIONE MODULARE



Cos'è una Funzione?

Una funzione è un blocco di codice con un nome, che può essere chiamato da qualsiasi punto del programma.

```
tipo_ritorno nome_funzione(tipo param1, tipo param2)
{
   istruzioni;
}
```

Una funzione = un compito. Il nome è un verbo in italiano che descrive cosa fa: stampa_menu, calcola_media, scambia.

Obiettivo	Che cosa significa
Suddividere	un problema complesso diventa una serie di compiti piccoli
Riutilizzare	lo stesso codice si scrive una volta e si chiama più volte
Isolare	ogni funzione ha una sola responsabilità

Definizione e Chiamata

```
void stampa_separatore(void)
{
    printf("--------------------\n");
}

int main(void)
{
    stampa_separatore();     /* chiamata */
    printf("Benvenuti\n");
    stampa_separatore();     /* stessa funzione, riusata */
    return 0;
}
```

Quando una funzione viene chiamata, l'esecuzione salta al suo corpo; al termine torna all'istruzione successiva alla chiamata.

Elemento	Esempio
tipo di ritorno	void, int, double, char
nome	stampa_separatore
parametri	(void) se non ce ne sono

Errore tipico: dimenticare le parentesi. stampa_separatore; senza () non chiama la funzione.

Il Prototipo

```
void stampa_titolo(void);       /* prototipo: termina con ; */

int main(void)
{
    stampa_titolo();            /* il compilatore conosce già la firma */
    return 0;
}

void stampa_titolo(void)        /* definizione completa */
{
    printf("=== Report ===\n");
}
```

Il compilatore legge il file dall'alto in basso: una funzione chiamata prima di essere definita è sconosciuta. Il prototipo la dichiara in anticipo.

Con i prototipi in cima al file, il main può stare subito dopo e si legge come un indice del programma.

Funzioni void

```
void stampa_intestazione(void)
{
    printf("TRATTORIA DA LUIGI\n");
    printf("Via Roma 12, Milano\n");
}

void stampa_menu_completo(void)
{
    stampa_intestazione();
    stampa_primi();
    stampa_dolci();
}
```

Le funzioni void non restituiscono un valore: producono effetti, come stampare a schermo.

- Una funzione può chiamarne altre: il programma si costruisce a livelli
- Per modificare una sezione si interviene in un solo punto

Una funzione void non si usa in un'espressione: x = stampa_intestazione(); è un errore.

Parametri per Valore

```
void raddoppia(int n)
{
    n = n * 2;          /* modifica solo la copia */
}

int x = 5;
raddoppia(x);
printf("%d\n", x);      /* stampa 5 */
```

I parametri sono variabili locali inizializzate con i valori passati nella chiamata. I tipi scalari (int, double, char) vengono passati per valore: la funzione riceve una copia.

Chiamante	Funzione
int x = 5	int n = 5 (copia)
x resta 5	n = 10, poi scompare

Modificare un parametro non cambia mai la variabile del chiamante.

Il Valore di Ritorno

```c
double media(int a, int b, int c)
{
    double risultato;
    risultato = (a + b + c) / 3.0;
    return risultato;          /* unica return, alla fine */
}
```

Il valore restituito si può usare:

- in un'assegnazione: m = media(7, 8, 6);
- in una printf: printf("%.2f", media(7, 8, 6));
- in un'espressione: totale = media(7, 8, 6) * 2;

Regola del corso: una sola return per funzione. Il risultato si accumula in una variabile locale e si restituisce alla fine.

Errore tipico: return multipli nei rami di un if. Si assegna il valore nei rami e si restituisce una volta sola.

Parametri Multipli e Responsabilità

```
double area_trapezio(double base_maggiore, double base_minore,
double altezza)
{
   double area;
   area = (base_maggiore + base_minore) * altezza / 2.0;
   return area;
}

a = area_trapezio(8.0, 5.0, 4.0);   /* l'ordine conta */
```

- Gli argomenti si abbinano ai parametri per posizione, non per nome
- Separazione delle responsabilità: le funzioni di calcolo restituiscono il risultato, non stampano; la stampa è compito del main o di funzioni dedicate

Firme uniformi: funzioni che fanno operazioni simili hanno gli stessi parametri nello stesso ordine.

Indirizzi e Puntatori

Ogni variabile occupa una cella di memoria con un indirizzo. Un puntatore è una variabile che contiene un indirizzo.

```
int eta = 25;
int *p = &eta;      /* p punta a eta */

*p = 30;            /* modifica eta tramite p */
printf("%d\n", eta); /* stampa 30 */
```

Operatore	Nome	Significato
&x	indirizzo-di	l'indirizzo di x
***p**	dereferenziazione	il valore all'indirizzo contenuto in p

In questo capitolo i puntatori servono solo per il passaggio per riferimento. Il capitolo 05 li tratta in modo completo.

Passaggio per Riferimento

Per modificare una variabile del chiamante, la funzione riceve il suo indirizzo.

```
void raddoppia(int *n)
{
   *n = *n * 2;        /* modifica la variabile puntata */
}

int x = 5;
raddoppia(&x);          /* si passa l'indirizzo */
printf("%d\n", x);      /* stampa 10 */
```

	Per valore	**Per riferimento**
chiamata	f(x)	f(&x)
parametro	int n	int *n
accesso	n	*n
effetto su x	nessuno	x viene modificata

Errore tipico: chiamare raddoppia(x) invece di raddoppia(&x). Il compilatore segnala il tipo sbagliato.

Parametri IN e OUT

Con il passaggio per riferimento una funzione può "restituire" più risultati.

```
void dividi(int a, int b, int *quoziente, int *resto)
{     /* IN     IN     OUT          OUT       */
    *quoziente = a / b;
    *resto = a % b;
}

dividi(17, 5, &q, &r);    /* q = 3, r = 2 */
```

Tipo	Uso	Passaggio
IN	dato in ingresso, solo letto	per valore
OUT	risultato scritto dalla funzione	per riferimento

Esempio classico: scambia(&a, &b). Con il passaggio per valore lo scambio sarebbe impossibile.

Variabili Locali

Una variabile dichiarata in una funzione o in un blocco { } è locale:

- è visibile solo in quel blocco
- viene creata quando l'esecuzione entra nel blocco e distrutta all'uscita
- non conserva il valore tra una chiamata e la successiva

```
void funzione_a(void)
{
   int x = 10;      /* x di funzione_a */
}

void funzione_b(void)
{
   int x = 99;      /* un'altra x, indipendente */
}
```

Stesso nome in funzioni diverse = variabili diverse, in celle di memoria diverse.

Variabili Globali

Una variabile dichiarata fuori da ogni funzione è globale:

- è visibile in tutte le funzioni del file
- esiste per tutta la durata del programma
- se non inizializzata vale 0

```
int errori = 0;          /* globale */

void segnala_errore(void)
{
   errori++;
}
```

Quando usarle	Quando evitarle
stato condiviso da più funzioni che deve persistere (contatori, min/max accumulati)	per evitare di passare parametri

Qualsiasi funzione può modificare una globale: il programma diventa più difficile da leggere e da verificare.

Shadowing

Una variabile locale con lo stesso nome di una globale la oscura nel proprio scope.

```
int valore = 100;        /* globale */

void funzione(void)
{
   int valore = 5;       /* locale: oscura la globale */
   valore++;             /* modifica la locale */
}                        /* la globale resta 100 */
```

```
int globale;             visibile in tutto il file
 void f(void) {
    int locale;          visibile solo in f
    {
       int interna;      visibile solo nel blocco
    }
 }
```

Lo shadowing è legale ma va evitato: si crede di modificare la globale e si modifica la copia locale. Usare nomi distinti.

Errori Comuni da Evitare

1

Chiamare una funzione prima di definirla senza prototipo.

2

Usare il valore di una funzione void o ignorare quello di una funzione che restituisce un risultato.

3

Aspettarsi che modificare un parametro cambi la variabile del chiamante.

4

Passare x invece di &x a una funzione che vuole un indirizzo.

5

Più return nella stessa funzione.

6

Variabile locale con lo stesso nome di una globale (shadowing).

Riepilogo Concetti

Operazione	Tecnica	Note
Suddividere il programma	funzioni void	una funzione = un compito
Chiamare prima di definire	prototipo	termina con ;
Fornire dati	parametri per valore	la funzione riceve una copia
Restituire un risultato	return	una sola, alla fine
Modificare il chiamante	parametro int *p	chiamata con &x
Più risultati	parametri OUT	IN per valore, OUT per riferimento
Dati temporanei	variabili locali	vivono solo nel blocco
Stato condiviso	variabili globali	con cautela
Nomi uguali	evitare lo shadowing	nomi distinti

SEZIONE 1

Esempi Pratici: Definizione e Chiamata

Concetti base — tre file sulle prime funzioni.

1 – 010-prima-funzione.c

Problema: definire e chiamare una funzione.

Tecnica: funzioni void senza parametri (benvenuto, separatore, arrivederci) chiamate dal main.

2 – 020-prototipo.c

Problema: chiamare una funzione definita più in basso.

Tecnica: prototipi in cima al file, main prima delle definizioni.

3 – 030-funzioni-void.c

Problema: organizzare un documento complesso.

Tecnica: funzioni void composte che chiamano altre funzioni.

SEZIONE 1

Esempi Pratici: Problemi su Definizione e Chiamata

040 – Menu ristorante

Una funzione per ogni sezione del menu; per cambiare i primi si modifica solo stampa_primi().

050 – Ricevuta di cassa

Intestazione, prodotti, riepilogo IVA e piede gestiti da funzioni separate.

060 – Disegno di forme

Rettangolo, triangolo e cornice in ASCII art, una funzione per forma.

SEZIONE 2

Esempi Pratici: Parametri e Return

Concetti base — tre file su ingresso e uscita dei dati.

1 – 010-parametri-per-valore.c

Problema: capire che cosa riceve una funzione.

Tecnica: copia del parametro, variabile del chiamante invariata, conversione di temperatura.

2 – 020-valore-di-ritorno.c

Problema: restituire un risultato.

Tecnica: return con int, double e char (somma, media, valore assoluto, maiuscolo).

3 – 030-parametri-multipli.c

Problema: funzioni con più dati in ingresso.

Tecnica: 2, 3 e 4 parametri; ordine degli argomenti; area del trapezio, massimo tra tre.

SEZIONE 2

Esempi Pratici: Problemi con Parametri e Return

040 – Calcolatrice

Una funzione per operazione; nessuna funzione di calcolo stampa; divisione per zero gestita.

050 – Conversione di unità

Menu con 10 conversioni di lunghezza, massa e temperatura, una funzione ciascuna.

060 – Analisi voti

Cinque voti come parametri: media, minimo, massimo, promozione e giudizio.

SEZIONE 3

Esempi Pratici: Passaggio per Riferimento

Concetti base — tre file su indirizzi e puntatori.

1 – 010-introduzione-ai-puntatori.c

Problema: che cos'è un indirizzo.

Tecnica: operatori & e *, modifica di una variabile tramite puntatore.

2 – 020-modifica-tramite-puntatore.c

Problema: modificare la variabile del chiamante.

Tecnica: raddoppia per valore e per riferimento a confronto; incrementa, azzera.

3 – 030-parametri-misti.c

Problema: restituire più risultati.

Tecnica: parametri IN e OUT; dividi, minimo e massimo, Celsius in Fahrenheit e Kelvin.

SEZIONE 3

Esempi Pratici: Problemi con il Passaggio per Riferimento

040 – Scambio di valori

scambia(&a, &b) con variabile temporanea; ordinamento di tre interi.

050 – Minimo e massimo

Cinque misurazioni: minimo e massimo in una sola chiamata, media ed escursione.

060 – Divisione intera

Secondi in ore, minuti e secondi; centesimi in euro e centesimi.

SEZIONE 4

Esempi Pratici: Scope e Visibilità

Concetti base — tre file sulla vita e sulla visibilità delle variabili.

1 – 010-variabili-locali.c

Problema: dove esiste una variabile.

Tecnica: stesso nome in funzioni diverse, blocchi annidati.

2 – 020-variabili-globali.c

Problema: condividere uno stato tra funzioni.

Tecnica: globale modificata da aggiungi, sottrai, azzera; inizializzazione a 0.

3 – 030-shadowing.c

Problema: nomi uguali in scope diversi.

Tecnica: una locale oscura la globale; effetti e come evitarlo.

SEZIONE 4

Esempi Pratici: Problemi su Scope e Visibilità

040 – Contatore di chiamate

Contatori globali per diagnosticare quante volte viene chiamata ogni funzione.

050 – Temperatura giornaliera

Stazione meteo: minimo e massimo globali aggiornati a ogni rilevazione.

060 – Validazione input

Una funzione per campo e un contatore globale di errori decide l'esito del modulo.